# LocalWebAuthn — The Examples

Perry Kundert

August 9, 2026

Companion to README.org. Five examples, each a complete and tested artifact rather than a snippet: the demo and the starter are exercised by the same Playwright and Vitest suites CI runs, and the three channel packages have their own tests.

They share one security stance, which is the reason to read them rather than invent equivalents:

- Enrollment links and signup proof codes ride URL **fragments**, so no server or proxy log ever sees them.
- Message content comes only from fixed templates; a caller passes a URL or a code, never copy.
- **Delivery is an internal function call.** No example exposes a "send email" or "send SMS" route, because anything that could reach such a route could spend your SMS balance and send DKIM-signed mail from your domain.

| You are... | Start with | It gives you |
|---|---|---|
| taking a first look at the whole lifecycle | `examples/demo` | Full UI: bootstrap, inv |
| wiring the six routes into your own server | `examples/starter-hono` | Headless Hono/Node: |
| sending email and SMS from a normal server | `examples/channels-node` | SMTP with an app pa |
| building with no server at all | `examples/channels-cf` | Workers + D1 issuing |
| looking for the parts both delivery shapes share | `examples/channels` | Templates, destination |

## 1 examples/demo — the whole lifecycle, with a UI

The one to run first. `@localwebauthn/server` with SQLite, `@localwebauthn/browser` for ceremonies, `@localwebauthn/client` **in the page** for minting API credentials, Hono for the HTTP and cookie adapter, Vite for assets, and one application-owned `demo_clients` table.

```
nix develop
make demo-reset   # optional: start from nothing
make demo
```

It binds to `127.0.0.1`, serves plain HTTP because localhost is the browser-sanctioned exception, and prints enrollment secrets to the terminal. Convenient for review; **not** a production template.

What it demonstrates, in the order you meet it:

1. Administrator bootstrap from a one-time enrollment URL.
2. Passkey-only sign-in and sign-out.
3. Administrator-created people and their enrollment links.
4. Enrollment in another browser profile or on another device.
5. A second passkey authorized by an existing session — no link.
6. Simulated self-serve signup and recovery: one proof link per channel shown in an on-screen inbox, cooperating proof pages, claim-on-reopen, and for existing accounts a waiting period with "this wasn't me" and the passkey-sign-in veto.
7. Session control: sign out everywhere for a person, or your own other devices, without touching passkeys.
8. Credential revocation, and whole-user re-enrollment in the documented order.
9. API credentials: mint one from the page, watch the private key appear exactly once, then run the real CLI script against it — and revoke it and watch the script stop.

Code map:

| File | Responsibility |
|---|---|
| `src/database.ts` | app-owned `demo_clients` plus the package's SQLite migration |
| `src/auth.ts` | the complete Hono adapter: origin check, cookies, six auth routes |
| `src/application.ts` | bootstrap, invite, re-enroll, revoke, session sign-out |
| `src/machine.ts` | `/api/api-keys/*` (browser-facing) and `/api/machine/v1/*` (script) |
| `src/client.ts` | the UI, through `LocalWebAuthnBrowser` — no raw WebAuthn calls |
| `scripts/api-client.ts` | the headless client; `--dry-run` prints every byte it constructs |
| `e2e/*.spec.ts` | Playwright with Chromium virtual passkeys |

The two files worth reading even if you never run it are `src/auth.ts`, which is the whole cookie and origin story in one place, and `src/machine.ts`, which shows why a script route and a browser route must have different shapes.

## 2 examples/starter-hono — the six routes, nothing else

No UI, no opinions. Copy `src/auth-routes.ts` into your application and adjust the cookie namespace.

```
nix develop
make starter-hono   # outside the shell: make nix-starter-hono
```

It prints a bootstrap enrollment URL and serves the six protocol routes, a session guard, the exact-origin middleware and an invite endpoint. `mountPasskeyAuth` must be called **before** your own `/api/*` routes, because its origin check only protects what is registered after it — the kind of ordering constraint that is obvious in code and invisible in prose.

## 3 examples/channels — what both delivery shapes share

No HTTP surface at all, and neither runtime example adds one.

| Piece | What it fixes |
|---|---|
| `templates.ts` | the only source of message content; callers pass a URL or code, never copy |
| `validate.ts` | E.164 plus an `SMS_ALLOWED_PREFIXES` country allowlist (SMS-pumping defence) |
| `twilio.ts` | fetch-based Twilio sender, identical under Node and Workers |
| `resend.ts` | fetch-based Resend sender, the Workers email transport |
| `deliver.ts` | `inviteAndDeliver`: issue the grant, deliver it, return **no link** |
| `signup.ts` | the self-serve proofing state machine |

Because content is template-only, the blast radius of a leaked provider credential or a buggy caller is "our own enrollment copy, at our rate" — a cost problem, never an arbitrary-content phishing relay from your domain or number.

`signup.ts` is the piece to read if you are designing self-serve onboarding. It implements capability-free proof links (the link proves a channel; it does

not grant anything), claim-on-reopen so the person finishes on the device they prefer, and — for accounts that already have passkeys, which is recovery rather than signup — a waiting period, vetoes from any channel, and cancellation on any successful passkey sign-in.

## 4 examples/channels-node — SMTP and Twilio from a server

```
import { createDelivery, inviteAndDeliver } from '@localwebauthn/channels-node';

const delivery = createDelivery(process.env);

// Inside your own authorized invite route:
const outcome = await inviteAndDeliver(auth, delivery, {
  userId,
  to: { email: 'person@example.com', phone: '+15551234567' },
});
return c.json({ delivered: outcome.delivered, expiresAt: outcome.expiresAt });
```

The outcome deliberately omits the URL and token: the only copy went to the destination. Email uses SMTP with an application-specific password through nodemailer, so no HTTP mail API credential is needed.

## 5 examples/channels-cf — fully Cloudflare

The Worker owns the users table and all LocalWebAuthn state in D1, issues real enrollment grants, and delivers them with Resend (Workers cannot speak SMTP) and Twilio.

There is no send API. The only public route is the application's own invite flow, which requires a bearer token standing in for your real session or admin authorization, fails closed if it is unset, sends only the fixed templates, and returns `{ delivered, expiresAt }` — never the link.

This is also the example to read for D1's constraint: no transactions, so the adapter batches and guards each statement on the preceding row count. What that does and does not guarantee is in SECURITY.md.

## 6 Running and testing them

```
nix develop
```

```
make test                  # unit, all three store adapters, and the channel examples
make test-demo-install     # once: Playwright Chromium
make test-demo             # the four browser specs
make check                 # typecheck, lint, format, coverage, package gates
```

Outside the flake shell, wrap any target: `make nix-test`, `make nix-check`. Anything under `examples/` imports `@localwebauthn/*` **by package name**, which resolves to each package's built `dist/` — so after changing package source, rebuild before expecting an example to see it:

```
npm run build --workspace @localwebauthn/server
```